

TP3 – Générateur de *memes* 🤡🤩

Le présent travail pratique a pour objectif de vous familiariser avec :

- la récupération de données depuis une API à l'aide de la méthode `fetch()`
- la gestion des *side effects* avec le *hook* `useEffect`
- l'utilisation de composants contrôlés pour une interactivité fluide et synchronisée entre l'interface utilisateur (UI) et les données internes

API *memes* à utiliser

Le travail consistera à mettre en place un générateur de *memes* en faisant usage de l'API suivante :

```
https://api.imgflip.com/get_memes
```

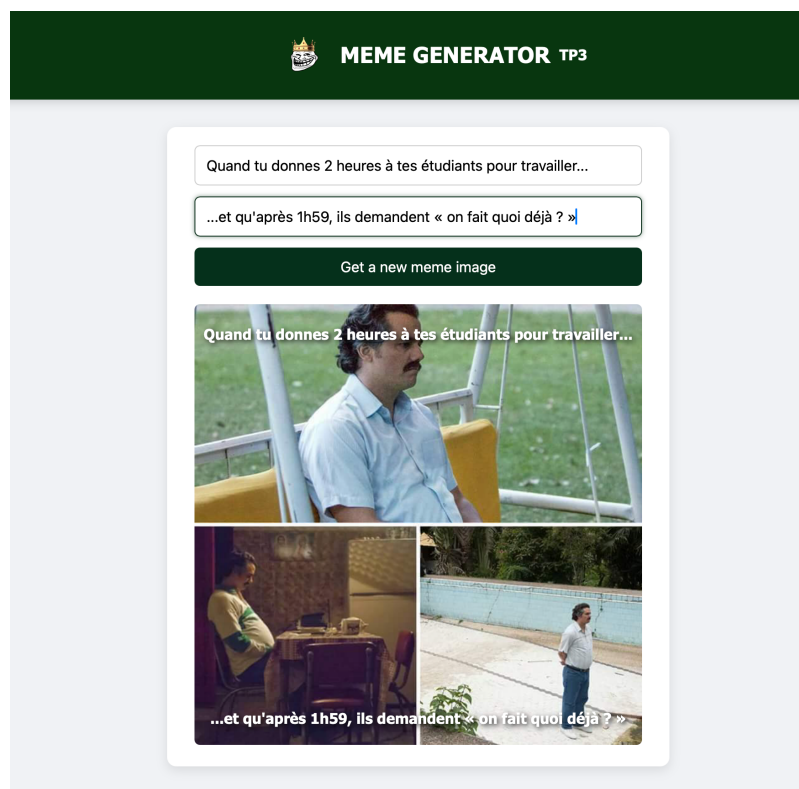
Cette API vous permettra de récupérer aléatoirement des images de *meme* que l'utilisateur pourra ensuite personnaliser avec du texte.

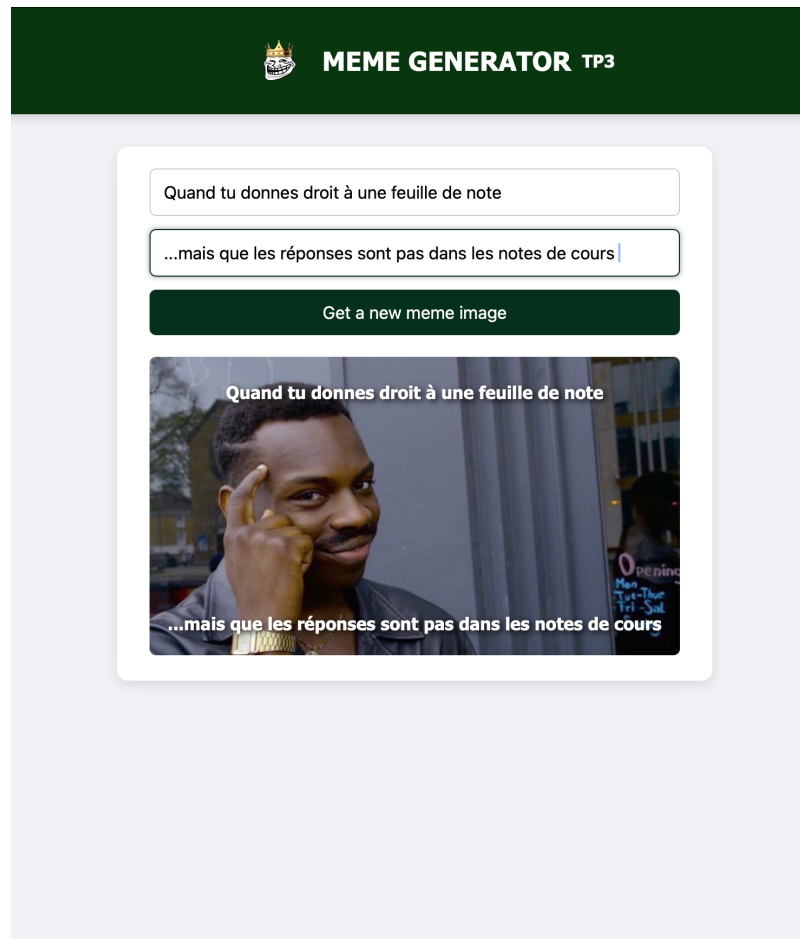
Indications détaillées

Votre application devra permettre à l'utilisateur :

- de générer un *meme* avec une image aléatoire **provenant de l'API** ci-dessus
- d'insérer du **texte personnalisé en haut et en bas** de l'image choisie
- de voir le texte s'afficher **en temps réel** à mesure qu'il est tapé dans les champs de saisie (utilisation des composants contrôlés)
- de récupérer une **nouvelle image de meme** à chaque clic sur le bouton prévu à cet effet

Voici quelques exemples de *memes* générés, illustrant le visuel attendu :





Vous êtes invités à être créatifs sur le *design*, rendez votre application originale et agréable visuellement.

La structure minimale attendue

Votre application React devra avoir minimalement :

- une en-tête (*header*)
- un corps (*main content*)
- un pied de page (*footer*)

Indications techniques

Vous devrez démarrer votre application à partir de zéro :

- aucun fichier HTML ou CSS ne vous sera fourni ; vous êtes libres de personnaliser entièrement votre application.
- votre structure de dossiers doit être claire :

```
src
├── components
│   ├── Header.jsx
│   ├── MemeGenerator.jsx
│   └── Footer.jsx
├── App.jsx
└── main.jsx
```

- toutes les images de *memes* doivent provenir directement de l'API (pas stockées localement).
- seules les images statiques d'interface (logo, icônes) peuvent être placées localement dans le dossier `/assets`.

Le meilleur *meme* 🎉

Question d'ajouter un peu de *fun*, chaque étudiant est invité à créer son meilleur *meme*, drôle et original. Insérez une capture d'écran de votre *meme* préféré dans votre remise. Le jour de la remise, nous les regarderons tous ensemble et voterons pour élire le *meme* le plus drôle ; soyez créatifs, soyez rigolos ! 😄

Les consignes de remise

Votre code doit être publié sur GitHub dans le dépôt (*repo*) principal des TPS

- créez une branche nommée TP3 pour votre travail
- assurez-vous de réaliser des commits fréquents avec des messages clairs et indicatifs de votre avancement

La date limite est le 24 avril avant minuit.

Grille de correction

Les critères d'évaluation	pondération
La présence d'un formulaire contenant deux champs de saisie (texte haut et bas), gérés en tant que composants contrôlés	2.5
La présence d'un bouton permettant de générer un <i>meme</i>	1.0
Utilisation de <code>fetch()</code> combinée à <code>useEffect</code> pour effectuer la récupération des données de l'API au chargement	3.0
La récupération correcte d'une image aléatoire à partir des données obtenues via l'API	1.0
Qualité du code : respect des bonnes pratiques enseignées (séparation en composants, utilisation de <code>state</code> , logique claire, etc.)	1.5
La présentation visuelle : qualité du rendu CSS (mise en page, lisibilité, responsivité minimale)	1.0
Non-respect des consignes de remise (mauvais dépôt, mauvaise branche, retard, etc.)	pénalité
Originalité / fonctionnalités supplémentaires (ajout pertinent de fonctionnalités et d'améliorations esthétiques)	bonus

/ 10